Contents

1 Searching Problems **3**
1.1 Problem Formulation . . . 3
1.1.1 Evaluation . . . 4
1.2 Uninformed Search . . . 4
1.2.1 Breadth-First Search (BFS) . . . 5
1.2.2 Uniform-Cost Search (UCS) . . . 6
1.2.3 Depth-First Search (DFS) . . . 7
1.2.4 Depth-Limited Search (DLS) . . . 8
1.2.5 Iterative-Deepening Search (IDS) . . . 9
1.2.6 Bidirectional Search . . . 10
1.3 Informed Search . . . 11
1.3.1 A* Search . . . 12
1.3.2 IDA* (Iterative-Deepening A*) . . . 14
1.3.3 Recursive Best-First Search (RBFS) . . . 15
1.4 Ricerca locale: cercare una configurazione . . . 16
1.4.1 Hill climbing e i suoi limiti . . . 16
1.4.2 Simulated annealing . . . 17
1.4.3 Beam search ed algoritmi evolutivi . . . 17
1.4.4 Spazi continui . . . 18
1.5 Azioni non deterministiche: piani condizionali . . . 18
1.5.1 Alberi AND-OR . . . 18
1.6 Osservazione incompleta: belief states . . . 19
1.6.1 Ambiente senza osservazioni . . . 19
1.6.2 Osservazione parziale: predict–percept–update . . . 20
1.7 Ricerca online: agire mentre si apprende . . . 20
1.7.1 Strategie online . . . 20
1.7.2 Learning Real-Time A* (LRTA*) . . . 21
1.8 Ricerca avversaria: giochi a somma zero . . . 21
1.8.1 Minimax . . . 22
1.9 Potatura alpha-beta: non esplorare ciò che non può servire . . . 23
1.10 Giocare con budget limitato: H-minimax . . . 23
1.10.1 Horizon effect e rimedi . . . 24
1.10.2 Monte Carlo Tree Search . . . 24
1.11 Giochi stocastici e giochi a informazione incompleta . . . 24
1.11.1 Chance nodes ed expectiminimax . . . 24
1.11.2 Informazione nascosta e averaging over clairvoyance . . . 25

1.12 Mappa finale per lo studio 25

1 Searching Problems

A searching problem can be formulated as a search for a path from an initial state to a goal state. This formulation assumes the environment is fully observable, deterministic, static, and discrete.

1.1 Problem Formulation

A search problem is defined as:

$$\boxed{\mathcal{P} = (S, s_0, G, A, T, c)}$$

where:

- S is the set of states.
- $s_0 \in S$ is the initial state.
- $G \subseteq S$ is the set of goal states.
- $A(s)$ is the set of applicable actions in state s.
- $T(s, a) \in S$, for $a \in A(s)$, is the successor state.
- $c(s, a, s')$ is the cost of transitioning from s to s' via a.

Assuming A is the global action set and T is restricted to applicable state-action pairs, ***DETERMINISM*** follows from the fact that:

$$T : S \times A \to S$$

is a single-valued function.

> ***Paths***

A path is a sequence:

$$\pi = (s_0, a_0, s_1, a_1, \dots, a_{n-1}, s_n)$$

such that:

$$\forall i \in \{0, \dots, n-1\} : \quad a_i \in A(s_i), \quad s_{i+1} = T(s_i, a_i)$$

A path is a solution if:

$$s_n \in G$$

Its total cost is:

$$\boxed{C(\pi) = \sum_{i=0}^{n-1} c(s_i, a_i, s_{i+1})}$$

A solution π^* is ***OPTIMAL*** if:

$$\boxed{\pi^* \in \arg\min_{\pi \in \Pi_G} C(\pi)}$$

where Π_G denotes the set of all solution paths.

1.1.1 Evaluation

A search algorithm $\mathcal{A}$ is evaluated according to four criteria:

- **Completeness:** whether $\mathcal{A}$ finds a solution whenever one exists.
- **Optimality:** whether $\mathcal{A}$ returns a minimum-cost solution.
- **Time Complexity:** number of nodes generated or expanded.
- **Space Complexity:** maximum number of nodes stored in memory.

We will use b for the branching factor, d for the depth of the shallowest solution, and m for the maximum depth of the search tree.

1.2 Uninformed Search

A search is uninformed if it uses only the problem definition, without any estimate of how close a state is to the goal.

The Python pseudocode uses the same abstract operations as the slides: `Node` creates a search node, `expand` generates its children, and `is_cycle` checks its ancestor path. Nodes store `state`, path cost `g` (initially zero), and `depth` (initially zero). `PriorityQueue` pops the node with the smallest key; `peek` reads it without removal. `None` means failure; `"cutoff"` means the depth limit was reached. Heuristics `h` take a node. For bidirectional search, `join_nodes` joins the two paths in forward order, and `terminated` checks the stopping condition for the chosen search strategy.

1.2.1 Breadth-First Search (BFS)

The algorithm expands nodes in order of increasing depth, exploring the shallowest nodes first. It uses a FIFO queue to store the frontier.

If the shallowest solution is at depth d, the number of nodes grows as:

$$1 + b + b^2 + \cdots + b^d = O(b^d)$$

> ***Pseudocode***

```python
from collections import deque


def bfs(problem):
    node = Node(problem.initial)
    if problem.is_goal(node.state):
        return node
    frontier, reached = deque([node]), {node.state}
    while frontier:
        for child in expand(problem, frontier.popleft()):
            if problem.is_goal(child.state):
                return child
            if child.state not in reached:
                reached.add(child.state)
                frontier.append(child)
    return None
```

> ***Evaluation***

Criterion	Value
Complete	Yes[1]
Optimal	Yes[3]
Space	$O(b^d)$
Time	$O(b^d)$
Advantages	finds shallowest solution
Disadvantages	high memory usage

Assumptions:

- (1) complete if b is finite, and the state space either has a solution or is finite.
- (3) cost-optimal if action costs are identical.

1.2.2 Uniform-Cost Search (UCS)

The algorithm expands nodes in order of increasing path cost, exploring the cheapest paths first. It uses a priority queue ordered by $g(n)$ to store the frontier.

It is equivalent to Dijkstra's algorithm when applied to shortest-path search with non-negative edge costs:

$$f(n) = g(n)$$

where $g(n)$ is the cost from the initial state to node n.

> ***Pseudocode***

```python
def best_first_search(problem, f):
    node = Node(problem.initial)
    frontier = PriorityQueue([node], key=f)
    reached = {node.state: node}
    while frontier:
        node = frontier.pop()
        if problem.is_goal(node.state):
            return node
        for child in expand(problem, node):
            s = child.state
            if s not in reached or child.g < reached[s].g:
                reached[s] = child
                frontier.add(child)
    return None


def ucs(problem):
    return best_first_search(problem, lambda n: n.g)
```

> ***Evaluation***

Criterion	Value
Complete	Yes[1,2]

Criterion	Value
Optimal	Yes
Space	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$
Time	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$
Advantages	finds the least-cost solution
Disadvantages	high time and memory usage

Assumptions:

- (1) complete if b is finite, and the state space either has a solution or is finite.
- (2) complete if all action costs are $\geq \varepsilon > 0$.

1.2.3 Depth-First Search (DFS)

The algorithm expands nodes in order of decreasing depth, following a branch as far as possible before backtracking. It uses a LIFO stack to store the frontier.

If b is the branching factor and m is the maximum depth, the number of nodes explored can grow exponentially:

$$1 + b + b^2 + \cdots + b^m = O(b^m)$$

> ***Pseudocode***

```
def dfs(problem):
    return dls(problem, float("inf"))
```

> ***Evaluation***

Criterion	Value
Complete	No
Optimal	No

Criterion	Value
Time	$O(b^m)$
Space	$O(bm)$
Advantages	low memory usage
Disadvantages	can get stuck in infinite branches

1.2.4 Depth-Limited Search (DLS)

The algorithm expands nodes in order of decreasing depth, following a branch up to a fixed depth limit ℓ before backtracking. It uses a LIFO stack to store the frontier.

The number of nodes explored is bounded by:

$$1 + b + b^2 + \cdots + b^\ell = O(b^\ell)$$

> ***Pseudocode***

```python
def dls(problem, limit):
    frontier = [Node(problem.initial)]
    result = None
    while frontier:
        node = frontier.pop()
        if problem.is_goal(node.state):
            return node
        if node.depth >= limit:
            result = "cutoff"
        elif not is_cycle(node):
            frontier.extend(expand(problem, node))
    return result
```

> ***Evaluation***

Criterion	Value
Complete	No
Optimal	No

Criterion	Value
Time	$O(b^{\ell})$
Space	$O(b\ell)$
Advantages	avoids infinite-depth exploration
Disadvantages	may miss solutions beyond the limit

1.2.5 Iterative-Deepening Search (IDS)

The algorithm repeatedly explores the search tree with increasing depth limits, restarting from the initial node until a solution is found. It uses depth-limited DFS at each iteration, storing only the current path and its unexplored alternatives.

The depth limits are:

$$\ell = 0, 1, 2, \dots, d$$

Although nodes are explored multiple times, the asymptotic time complexity remains:

$$1 + b + b^2 + \cdots + b^d = O(b^d)$$

> ***Pseudocode***

```python
from itertools import count


def ids(problem):
    for depth in count():
        result = dls(problem, depth)
        if result != "cutoff":
            return result
```

> ***Evaluation***

Criterion	Value
Complete	Yes[1]
Optimal	Yes[3]
Time	$O(b^d)$
Space	$O(bd)$
Advantages	low memory usage
Disadvantages	repeated exploration of shallow nodes

Assumptions:

- (1) complete if b is finite, and the state space either has a solution or is finite.
- (3) cost-optimal if action costs are identical.

1.2.6 Bidirectional Search

The algorithm searches forward from the initial state and backward from the goal, looking for a state reached by both searches. It uses a separate frontier for each direction, with a FIFO queue for each when both searches use BFS.

Each search explores approximately half the solution depth:

$$b^{d/2} + b^{d/2} \ll b^d$$

It is applicable when the goal state is known, backward transitions can be generated, and the intersection between the two frontiers can be detected efficiently.

> ***Pseudocode***

```
def bidirectional_search(p_f, f_f, p_b, f_b):
    n_f, n_b = Node(p_f.initial), Node(p_b.initial)
    q_f = PriorityQueue([n_f], key=f_f)
    q_b = PriorityQueue([n_b], key=f_b)
    r_f, r_b = {n_f.state: n_f}, {n_b.state: n_b}
    solution = None
    if n_f.state == n_b.state:
        return n_f
    while not terminated(solution, q_f, q_b):
        if f_f(q_f.peek()) < f_b(q_b.peek()):
```

```
            solution = proceed("F", p_f, q_f, r_f, r_b, solution)
        else:
            solution = proceed("B", p_b, q_b, r_b, r_f, solution)
    return solution


def proceed(direction, problem, frontier, reached, other, solution):
    for child in expand(problem, frontier.pop()):
        s = child.state
        if s not in reached or child.g < reached[s].g:
            reached[s] = child
            frontier.add(child)
            if s in other:
                candidate = join_nodes(direction, child, other[s])
                if solution is None or candidate.g < solution.g:
                    solution = candidate
    return solution
```

> ***Evaluation***

Criterion	Value
Complete	Yes[1,4]
Optimal	Yes[3,4]
Space	$O(b^{d/2})$
Time	$O(b^{d/2})$
Advantages	much faster than BFS for large d
Disadvantages	high memory usage

Assumptions:

- (1) complete if b is finite, and the state space either has a solution or is finite.
- (3) cost-optimal if action costs are identical.
- (4) if both directions are breadth-first or uniform-cost.

1.3 Informed Search

A search is informed if it uses a heuristic function:

$$h : S \to \mathbb{R}_{\geq 0}$$

to guide the search toward promising states, reducing unnecessary exploration.

> ***Admissible Heuristics***

Let $h^*(s)$ be the true minimum cost from s to a goal.

A heuristic is ***ADMISSIBLE*** if it never overestimates the actual cost:

$$\boxed{\forall s \in S: \quad 0 \leq h(s) \leq h^*(s)}$$

An admissible heuristic is always optimistic, but not necessarily accurate.

1.3.1 A* Search

The algorithm expands nodes in order of increasing estimated total cost, exploring the most promising paths first. It uses a priority queue ordered by $f(n)$ to store the frontier.

The estimated total cost is:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ is the cost from the initial state to node n.
- $h(n)$ is the heuristic estimate from n to a goal.
- $f(n)$ is the estimated total cost of a solution through n.

> ***Pseudocode***

```
def a_star(problem, h):
    return best_first_search(problem, lambda n: n.g + h(n))
```

> ***Evaluation***

Criterion	Value
Complete	Yes
Optimal	Yes[1]

Criterion	Value
Advantages	heuristic-guided exploration
Disadvantages	high memory usage

Assumptions:

- (1) optimal if h is admissible.

> ***PROOF (1)***

Assume h is admissible and A* returns a suboptimal solution of cost $C > C^*$, where C^* is the optimal cost.

Before selecting that suboptimal goal, there must be a node n on an optimal path still in the frontier, reached with its optimal cost $g^*(n)$.

Since A* selects the node with minimum f, selecting the suboptimal goal implies:

$$C \leq f(n)$$

However, by definition:

$$f(n) = g^*(n) + h(n)$$

Since h is admissible:

$$h(n) \leq h^*(n)$$

Therefore, because n lies on an optimal path:

$$f(n) \leq g^*(n) + h^*(n) = C^*$$

Combining the inequalities:

$$\boxed{C \leq f(n) \leq C^* < C}$$

This is ***IMPOSSIBLE***.

Hence A* cannot return a suboptimal solution under the stated assumptions.

$$\boxed{C = C^*}$$

1.3.2 IDA* (Iterative-Deepening A*)

The algorithm repeatedly explores the search tree with increasing cost limits, skipping nodes whose estimated total cost exceeds the current limit. It uses depth-first search at each iteration, storing only the current path and its unexplored alternatives.

The estimated total cost is:

$$f(n) = g(n) + h(n)$$

If no solution is found, the cost limit increases to the smallest f-value that exceeded the previous limit.

> ***Pseudocode***

```python
def ida_star(problem, h):
    def search(node, limit):
        f = node.g + h(node)
        if f > limit:
            return None, f
        if problem.is_goal(node.state):
            return node, f
        next_limit = float("inf")
        if not is_cycle(node):
            for child in expand(problem, node):
                result, cost = search(child, limit)
                if result is not None:
                    return result, cost
                next_limit = min(next_limit, cost)
        return None, next_limit

    root = Node(problem.initial)
    limit = h(root)
    while limit < float("inf"):
        result, limit = search(root, limit)
        if result is not None:
            return result
    return None
```

> ***Evaluation***

Criterion	Value
Complete	Yes
Optimal	Yes[1]
Advantages	low memory usage
Disadvantages	high time usage

Assumptions:

- (1) optimal if h is admissible.

1.3.3 Recursive Best-First Search (RBFS)

The algorithm follows the path with the lowest estimated total cost, backtracking when another path becomes more promising. It uses recursive depth-first search, remembering the best alternative cost at each level to decide when to backtrack.

> ***Pseudocode***

```
def rbfs(problem, h):
    def search(node, limit):
        if problem.is_goal(node.state):
            return node, node.f
        successors = list(expand(problem, node))
        if not successors:
            return None, float("inf")
        for child in successors:
            child.f = max(child.g + h(child), node.f)
        while True:
            successors.sort(key=lambda n: n.f)
            best = successors[0]
            if best.f > limit or best.f == float("inf"):
                return None, best.f
            alternative = (successors[1].f if len(successors) > 1
                           else float("inf"))
            result, best.f = search(best, min(limit, alternative))
            if result is not None:
                return result, best.f

```

```
    root = Node(problem.initial)
    root.f = h(root)
    return search(root, float("inf"))[0]
```

> ***Evaluation***

Criterion	Value
Complete	Yes
Optimal	Yes[1]
Advantages	low memory usage
Disadvantages	high time usage

Assumptions:

- (1) optimal if h is admissible.

1.4 Ricerca locale: cercare una configurazione

La ricerca dei capitoli precedenti cerca un **piano**, cioè una sequenza di azioni. Alcuni problemi, invece, chiedono direttamente una configurazione che soddisfi un vincolo o ottimizzi un criterio: nel problema delle n regine, per esempio, interessa una disposizione della scacchiera, non la sequenza con cui le regine sono state collocate.

Nella **ricerca locale** gli stati sono configurazioni vicine tra loro e si massimizza una funzione obiettivo $V : S \to \mathbb{R}$. Se $c(s)$ è il numero di conflitti, un possibile obiettivo per le n regine è $V(s) = -c(s)$. Di solito si conserva solo la configurazione corrente, non l'intero cammino: si risparmia memoria, ma non si ottiene automaticamente una soluzione ottima.

1.4.1 Hill climbing e i suoi limiti

L'algoritmo passa allo stato vicino con il valore obiettivo più alto, fermandosi quando nessun vicino migliora lo stato corrente. Mantiene solo lo stato corrente e valuta i suoi vicini a ogni passo.

La regola di aggiornamento è:

$$s \leftarrow \arg\max_{s' \in N(s)} V(s') \qquad \text{se } V(s') > V(s).$$

è semplice e utile anche come componente di algoritmi più complessi, ma può arrestarsi in:

- un **massimo locale**, migliore dei soli vicini ma non globale;
- un **plateau**, regione in cui i vicini hanno lo stesso valore;
- una **cresta** (*ridge*), lungo la quale un miglioramento richiede una sequenza di mosse non immediatamente vantaggiose.

Varianti importanti:

- **sideways moves**: permette mosse con valore invariato, per attraversare un plateau;
- **stochastic hill climbing**: sceglie casualmente fra vicini buoni;
- **first-choice**: prova vicini casuali finchè trova il primo miglioramento;
- **random-restart**: dopo un blocco ricomincia da una configurazione casuale e restituisce il miglior stato visto in tutti i tentativi.

1.4.2 Simulated annealing

L'algoritmo sceglie un vicino casuale, accettando sempre i miglioramenti e talvolta anche le mosse peggiorative. Mantiene solo lo stato corrente e usa una temperatura decrescente per ridurre gradualmente la probabilità di accettare peggioramenti.

Se $\Delta = V(s') - V(s)$, una regola tipica è

$$\Pr(\text{accettare } s') = \begin{cases} 1 & \text{se } \Delta \geq 0, \\ e^{\Delta/T} & \text{se } \Delta < 0, \end{cases}$$

dove $T > 0$ è la **temperatura**. All'inizio T è alta, quindi anche mosse peggiori sono abbastanza probabili e la ricerca esplora. Riducendo gradualmente T, l'algoritmo diventa sempre più simile a hill climbing: sfrutta le zone già promettenti.

1.4.3 Beam search ed algoritmi evolutivi

L'algoritmo di **beam search** genera i vicini degli stati correnti e seleziona i k migliori per il passo successivo. Mantiene un insieme di $k > 1$ stati, condividendo la selezione tra tutti i vicini generati. Questo fa comunicare informazioni tra i diversi filoni di ricerca; se però gli stati diventano troppo simili, si riduce a un hill climbing più costoso. Anche qui esistono versioni stocastiche.

L'**algoritmo evolutivo** genera nuove configurazioni combinando e modificando quelle selezionate in base alla fitness. Mantiene una popolazione di configurazioni, aggiornata a ogni generazione.

Il ciclo è:

1. genera una popolazione iniziale casuale;
2. seleziona le configurazioni migliori come genitori;
3. applica **crossover** per produrre figli;
4. applica con piccola probabilità una **mutazione** casuale;
5. opzionalmente conserva direttamente i migliori genitori (**elitismo**).

L'intuizione è combinare parti buone di soluzioni diverse. La scelta della rappresentazione e dell'operatore di crossover è decisiva: deve poter preservare contributi utili dei genitori, non solo mescolarli senza significato.

1.4.4 Spazi continui

Se uno stato è un vettore $x = (x_1, \dots, x_n) \in \mathbb{R}^n$ e l'obiettivo $f(x)$ è sufficientemente regolare, si possono usare strumenti di analisi, come gradient descent/ascent o Newton–Raphson. In presenza di vincoli regolari si entra nell'ottimizzazione vincolata; il caso lineare è la programmazione lineare, risolvibile in tempo polinomiale (per esempio con il metodo del simplesso).

1.5 Azioni non deterministiche: piani condizionali

Se un'azione non determina un unico stato successivo, la funzione di transizione diventa

$$\mathrm{RESULTS}(s, a) \subseteq S, \qquad \mathrm{RESULTS}(s, a) \neq \emptyset.$$

Non basta più scegliere una sola sequenza di azioni: il piano deve specificare cosa fare per ogni possibile esito. Per esempio, un'azione `suck` nel Vacuum World può condurre a stati diversi; un piano può avere la forma

$$[\texttt{suck},\ \text{if state} = 5 \text{ then } [\texttt{right}, \texttt{suck}] \text{ else } []].$$

1.5.1 Alberi AND-OR

Gli **OR-node** rappresentano la scelta dell'agente: per risolverli è sufficiente scegliere almeno un successore risolvibile. Gli **AND-node** rappresentano gli esiti dell'ambiente: sono risolti solo se *tutti* i loro successori sono risolvibili.

Formalmente, una soluzione è un sottoalbero tale che:

1. contiene la radice;

2. ogni sua foglia è un goal;
3. per ogni OR-node contiene almeno un successore;
4. per ogni AND-node contiene tutti i successori.

Le strategie deterministiche (DFS, BFS, euristiche, A*) si possono adattare. In un AND-node si costruisce un piano per ogni esito e li si combina in un piano condizionale:

$$[\text{if } s_1 \text{ then } p_1 \text{ else if } s_2 \text{ then } p_2 \ \cdots \ \text{else if } s_k \text{ then } p_k].$$

Gli alberi possono essere infiniti; durante la ricerca i cicli vengono trattati come fallimenti del ramo, poichè un buon piano non ha bisogno di attraversarli.

1.6 Osservazione incompleta: belief states

Quando l'agente non conosce esattamente lo stato fisico, ragiona su un **belief state** $b \subseteq S$: l'insieme degli stati che ritiene possibili. Questo trasforma l'incertezza in uno spazio di ricerca ordinario, ma più grande.

1.6.1 Ambiente senza osservazioni

Nel caso estremo, l'agente è senza sensori e deve trovare una sequenza che funzioni indipendentemente dallo stato iniziale reale. La formulazione in belief states è:

- stati: sottoinsiemi $b \subseteq S$;
- stato iniziale: l'insieme degli stati fisici inizialmente possibili;
- azioni legali (assumendo che un'azione illegale non faccia nulla):

$$A(b) = \bigcup_{s \in b} A(s);$$

- transizione deterministica:

$$T(b, a) = \{s' \mid \exists s \in b,\ s' = T(s, a)\};$$

- goal: b è un belief goal se $b \subseteq G$.

Con N stati fisici esistono fino a 2^N belief states. Non tutti sono raggiungibili, e si può fare pruning: se un piano risolve b, allora risolve anche ogni $b' \subseteq b$. Per problemi grandi, rappresentare esplicitamente un insieme di stati può essere impraticabile; una rappresentazione logica può essere più compatta.

1.6.2 Osservazione parziale: predict–percept–update

Con osservazioni parziali, dopo un'azione il percetto ricevuto restringe il belief state. Dato b e a, il ciclo è:

$$\begin{aligned}\mathrm{Predict}(b,a) &= \hat{b} = \{T(s,a) \mid s \in b\},\\ \mathrm{PossiblePercepts}(\hat{b}) &= \{o \mid \exists s \in \hat{b},\ o = \mathrm{Percept}(s)\},\\ \mathrm{Update}(\hat{b},o) &= \{s \in \hat{b} \mid \mathrm{Percept}(s) = o\}.\end{aligned}$$

L'azione diventa quindi non deterministica a livello dei belief states, perchè l'agente non sa quale percetto riceverà:

$$\mathrm{RESULTS}(b,a) = \{\mathrm{Update}(\mathrm{Predict}(b,a),o) \mid o \in \mathrm{PossiblePercepts}(\mathrm{Predict}(b,a))\}.$$

Si può dunque applicare una AND-OR search: l'agente sceglie l'azione, mentre l'ambiente determina il percetto/esito.

1.7 Ricerca online: agire mentre si apprende

Finora l'agente calcolava l'intero piano e poi agiva. Nella **ricerca online** l'ambiente è ignoto: l'agente deve alternare computazione, percezione e azione. Nel modello della lezione l'ambiente è deterministico e osservabile, ma il costo $c(s,a,s')$ e lo stato risultante s' sono noti solo dopo aver provato l'azione.

La qualità si misura con il **competitive ratio**:

$$\frac{\text{costo totale del cammino percorso}}{\text{costo del cammino ottimo}}.$$

L'inefficienza non è eliminabile: anche se tutte le azioni sono reversibili, un ambiente sconosciuto può costringere qualunque agente online a esplorazioni arbitrariamente costose. Se esistono azioni irreversibili, l'agente può inoltre entrare in un dead end da cui non è possibile uscire.

1.7.1 Strategie online

- **Online DFS**: prova azioni non ancora esplorate e torna indietro quando serve. Se le azioni sono reversibili e il goal è raggiungibile, lo trova prima o poi; il suo competitive ratio può però essere arbitrariamente cattivo.

- **Random walk**: sceglie casualmente un'azione non provata, se disponibile, altrimenti una qualunque azione. Con azioni reversibili raggiunge eventualmente il goal raggiungibile, ma il costo può essere esponenziale.
- **Hill climbing online**: un'euristica può guidarlo verso il goal, ma non può fare random restart quando incontra un minimo locale, perchè non può ricominciare liberamente dall'ambiente reale.

1.7.2 Learning Real-Time A* (LRTA*)

L'algoritmo sceglie l'azione con il minor costo stimato per raggiungere il goal, aggiornando le stime mentre esplora l'ambiente. Mantiene una tabella $H(s)$, inizializzata con l'euristica $h(s)$, per ricordare il costo stimato da ogni stato al goal. Dopo aver osservato la transizione eseguita, aggiorna lo stato precedente:

$$H(s) \leftarrow \min_{a \in A(s)} \mathrm{LRTA^*\text{-}Cost}(s, a, T[s, a], H),$$

dove $\mathrm{LRTA^*\text{-}Cost}(s, a, s', H)$ restituisce $h(s')$ se il successore s' non è ancora noto; altrimenti restituisce $c(s, a, s') + H(s')$.

Poi sceglie l'azione che minimizza lo stesso costo stimato. Aggiornando H, LRTA* impara che certe zone apparentemente buone sono vicoli ciechi o minimi locali e può quindi uscirne.

1.8 Ricerca avversaria: giochi a somma zero

Finora l'ambiente non aveva obiettivi propri. Nei giochi, altri agenti scelgono azioni che possono essere in conflitto con quelle dell'agente. Il caso base è un gioco a due giocatori, deterministico, a informazione perfetta, a lunghezza finita e **a somma zero**: ciò che è un guadagno per MAX è una perdita per MIN.

Un modello di gioco contiene:

- uno stato iniziale s_0;
- $\mathrm{TO\text{-}MOVE}(s)$, il giocatore che deve muovere in s;
- $A(s)$, le mosse legali;
- $T(s, a)$, lo stato dopo la mossa a;
- $\mathrm{IS\text{-}TERMINAL}(s)$, che riconosce la fine della partita;
- $\mathrm{UTILITY}(s)$, il punteggio finale di uno stato terminale.

1.8.1 Minimax

L'algoritmo sceglie la mossa con il miglior risultato garantito, assumendo che l'avversario risponda sempre nel modo migliore per sé. Usa una visita ricorsiva dell'albero di gioco, propagando i valori dalle foglie con il massimo nei nodi MAX e il minimo nei nodi MIN.

Il valore minimax si calcola dal basso verso l'alto:

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s), & \text{se IS-TERMINAL}(s), \\ \max\limits_{a \in A(s)} \text{MINIMAX}(T(s,a)), & \text{se TO-MOVE}(s) = \text{MAX}, \\ \min\limits_{a \in A(s)} \text{MINIMAX}(T(s,a)), & \text{se TO-MOVE}(s) = \text{MIN}. \end{cases}$$

MAX sceglie dunque una mossa che massimizza il valore, mentre MIN ne sceglie una che lo minimizza. Con branching factor medio b e profondità media m, minimax richiede tempo $O(b^m)$: per gli scacchi, dove approssimativamente $b \approx 35$ e $m \approx 80$, la ricerca esaustiva è impraticabile. Lo spazio è $O(bm)$, riducibile a $O(m)$ con una visita depth-first accurata.

Pseudocodice da saper ricostruire. La funzione principale restituisce la mossa con valore maggiore; le due funzioni ricorsive restituiscono la coppia `(valore, mossa)`:

```
MINIMAX(pos):
    valore, mossa = MAX-VALUE(pos)
    restituisci mossa

MAX-VALUE(pos):
    se TERMINAL(pos): restituisci (UTILITY(pos), null)
    migliore = (-infinito, null)
    per ogni mossa in ACTIONS(pos):
        valore, _ = MIN-VALUE(RESULT(pos, mossa))
        se valore > migliore.valore: migliore = (valore, mossa)
    restituisci migliore

MIN-VALUE(pos):
    se TERMINAL(pos): restituisci (UTILITY(pos), null)
    migliore = (+infinito, null)
    per ogni mossa in ACTIONS(pos):
        valore, _ = MAX-VALUE(RESULT(pos, mossa))
        se valore < migliore.valore: migliore = (valore, mossa)
    restituisci migliore
```

Con più di due giocatori si può usare una funzione $\text{UTILITY}(s, p)$ per ciascun giocatore p e mantenere i punteggi di tutti. La stessa idea si estende, ma possono emergere alleanze tra giocatori.

1.9 Potatura alpha-beta: non esplorare ciò che non può servire

L'algoritmo calcola la stessa scelta di minimax, interrompendo l'esplorazione dei rami che non possono cambiare la decisione. Usa una visita ricorsiva dell'albero di gioco e due limiti, α e β, per decidere quando interrompere un ramo.

I due limiti sono:

- α: il valore più alto che MAX può già garantire;
- β: il valore più basso che MIN può già garantire.

Inizialmente $[\alpha, \beta] = [-\infty, +\infty]$. In un MAX-node si aggiorna α con il massimo valore visto; in un MIN-node si aggiorna β con il minimo valore visto. Quando $\alpha \geq \beta$, il resto del sottoalbero può essere scartato: il giocatore che muove non lo sceglierebbe comunque. Questa è una **potatura**, non un'approssimazione.

L'ordine delle mosse è cruciale:

- con ordine ideale (prima le mosse migliori), il tempo scende a $O(b^{m/2})$;
- con ordine casuale, nelle slide la stima tipica è $O(b^{3m/4})$;
- criteri specifici del gioco o dati di partite possono suggerire mosse promettenti e migliorare la potatura.

1.10 Giocare con budget limitato: H-minimax

L'algoritmo sceglie la mossa con il miglior valore minimax stimato, fermando l'esplorazione a una profondità di cutoff. Usa una visita ricorsiva dell'albero di gioco e una funzione euristica $\mathrm{EVAL}(s)$ per valutare le posizioni al cutoff.

Per gli stati terminali si impone $\mathrm{EVAL}(s) = \mathrm{UTILITY}(s)$.

La ricorrenza di **H-minimax** è minimax, eccetto per il caso di cutoff:

$$\text{H-MINIMAX}(s, d) = \mathrm{EVAL}(s) \qquad \text{se IS-CUTOFF}(s, d).$$

Negli scacchi, una euristica base può sommare il valore dei pezzi:

$$\mathrm{EVAL}(s) = \#\text{pedoni} + 3\#\text{cavalli} + 3\#\text{alfieri} + 5\#\text{torri} + 9\#\text{regine}.$$

è solo una stima: il valore di un pezzo dipende dalla posizione e dalla fase della partita.

1.10.1 Horizon effect e rimedi

Un cutoff puramente per profondità può provocare l'**horizon effect**: se una perdita inevitabile cade appena oltre l'orizzonte di ricerca, il programma preferisce mosse che la rimandano oltre il cutoff, anche quando peggiorano ulteriormente la posizione.

Due rimedi presentati sono:

- interrompere la ricerca solo in posizioni **quiescenti**, cioè senza mosse imminenti che cambierebbero molto la valutazione;
- usare **singular extensions**: oltre il cutoff si esplorano soltanto mosse chiaramente migliori delle alternative.

Si possono inoltre fare **forward pruning** (considerare solo le mosse più promettenti, oppure approfondire meno le altre) e usare **lookup tables** per aperture, finali o posizioni ricorrenti già risolte.

1.10.2 Monte Carlo Tree Search

L'algoritmo esplora le mosse attraverso simulazioni ripetute, concentrando progressivamente la ricerca sulle alternative più promettenti. Mantiene un albero con il numero di visite e i risultati delle simulazioni per guidare le esplorazioni successive. L'idea è esplorare poche alternative in profondità, invece di moltissime alternative in modo superficiale.

1.11 Giochi stocastici e giochi a informazione incompleta

1.11.1 Chance nodes ed expectiminimax

L'algoritmo sceglie la mossa con il miglior valore atteso, considerando sia le risposte dell'avversario sia gli esiti casuali. Usa una visita ricorsiva dell'albero di gioco, calcolando il massimo nei nodi MAX, il minimo nei nodi MIN e la media pesata nei nodi CHANCE.

Nei giochi con casualità, come backgammon o Yahtzee, **CHANCE** (o Nature) sceglie un esito r con probabilità $P(r)$. La ricorrenza di **expectiminimax** conserva i casi MAX e MIN di minimax e aggiunge:

$$\mathrm{EXMINIMAX}(s) = \sum_{r \in A(s)} P(r) \cdot \mathrm{EXMINIMAX}(T(s,r)) \qquad \text{se } \mathrm{TO\text{-}MOVE}(s) = \mathrm{CHANCE}.$$

Le idee di alpha-beta, euristiche e forward pruning si applicano anche qui. Per potare nodi casuali servono limiti superiore e inferiore dell'utilità attesa; inoltre non conta solo l'ordine degli esiti, ma anche

la loro probabilità. La casualità ingrandisce fortemente l'albero: due dadi, per esempio, moltiplicano per 36 il numero di esiti a ogni turno.

1.11.2 Informazione nascosta e averaging over clairvoyance

L'algoritmo valuta ogni mossa risolvendo separatamente gli stati reali possibili, come se fossero completamente osservabili. Usa la media dei valori ottenuti per stimare il valore della mossa in presenza di informazione nascosta.

Il limite è fondamentale: questo metodo non rappresenta lo **stato informativo** del giocatore. Per esempio, non distingue il caso in cui un premio è a sinistra da quello in cui è a destra, se il giocatore non può osservarlo; per lo stesso motivo non modella bene il bluff. Può essere utile come approssimazione, ma il modello corretto usa belief states, al costo di una crescita molto forte di tempo e memoria.

1.12 Mappa finale per lo studio

1. **Spazio degli stati**: modella stati, azioni, transizione e costi; scegli poi la strategia di esplorazione.
2. **Ricerca classica**: senza euristica, iterative deepening è spesso il compromesso pratico; con un'euristica ammissibile, A* è ottima. Se la memoria è il limite, usa IDA* o RBFS.
3. **Ricerca locale**: cerca una configurazione, non un piano. Hill climbing è economico ma può bloccarsi; simulated annealing, beam search ed evoluzione introducono esplorazione e diversità.
4. **Azioni non deterministiche**: la soluzione è un piano condizionale; gli OR-node sono scelte dell'agente, gli AND-node sono esiti che vanno tutti gestiti.
5. **Informazione incompleta**: sostituisci lo stato fisico con un belief state. Con osservazioni parziali applica il ciclo predict–percept–update e una ricerca AND-OR.
6. **Ambiente ignoto**: nella ricerca online si agisce mentre si apprende; l'inefficienza è inevitabile. LRTA* aggiorna l'euristica memorizzata per evitare di insistere sui minimi locali.
7. **Giochi avversari**: minimax sceglie il massimo valore garantibile contro MIN; alpha-beta conserva la stessa scelta eliminando rami irrilevanti.
8. **Giochi complessi**: H-minimax usa una valutazione euristica al cutoff; expectiminimax sostituisce ai nodi casuali la media pesata degli esiti. Con informazione nascosta, il riferimento corretto resta il belief state.